

End-of-Term Project: E-Commerce Data Pipeline

Overview

In this project, you will design and document a complete data pipeline that collects product data from real-world e-commerce APIs and processes it across multiple database systems.

Your pipeline will:

1. Fetch product data from Shopify stores (JSON API)
2. Store and query the data in MongoDB
3. Transform and load the data into PostgreSQL
4. Model relationships in Neo4j
5. Generate a structured XML report

The primary focus is on your understanding, design decisions, and interpretation of results, which you will present in a written report.

Dataset

You must use the Shopify Products API from **at least two different online stores** (e.g. `.../products.json`).

Important Requirements

- You must handle **pagination** (Shopify limits results per request)
- Some stores may block or restrict access. If so:
 - Document the issue briefly
 - Use an alternative store
- You must clearly identify which stores you used

Learning Outcomes

You will demonstrate your ability to:

- Connect to a remote server using SSH and manage sessions with `tmux`
- Write Python scripts to fetch and process JSON data
- Insert and query data in MongoDB, including indexing and aggregation
- Design and populate a relational schema in PostgreSQL with transactions
- Model and query relationships in Neo4j
- Generate well-formed and structured XML
- Explain infrastructure and security considerations

Submission Requirements

1-Submit a **single ZIP file** containing:

File	Description
Final_Report.pdf	Main report (see structure below)
load_products.py	MongoDB ingestion script
mongo_to_pg.py	PostgreSQL transfer script
generate_report.py	XML generation script
mongo_queries.txt	MongoDB queries
postgres_queries.sql	SQL queries
cypher_queries.cyp	Neo4j queries
store_report.xml	Generated XML output

2-Submit Final_Report.pdf only on the separate Turnitin link

Notes

- Scripts must be functional and clearly commented

Report Structure (3000 - 4000 words max, excluding appendices)

1. Environment and Tools (10%)

- Explain how you connected via SSH
- Describe how you used tmux (multiple panes/sessions)
- Outline your Python setup (virtual environment and packages)
- Include a screenshot showing:
 - mongosh
 - psql
 - ipython3 running in tmux

2. MongoDB: Data Ingestion and Querying (15%)

- Describe your collections (e.g. stores, products)
- Explain how your script:
 - Fetches API data
 - Handles errors and pagination to retrieve all products from the API.
- Define your **compound unique index** (e.g. store + product ID) and justify it

Provide **three queries**, including:

- Query syntax
- Output (screenshot or formatted text)
- Interpretation of what the result shows in your dataset

3. PostgreSQL: Schema and Transactions (15%)

- Present your schema (must include **at least two related tables**, e.g. stores and products)
- Justify:
 - Data types
 - Use of primary keys (e.g. SERIAL)
 - Foreign key relationships
- Explain your transaction block and how it ensures **atomicity and consistency**

Important Requirements: The transfer script must explicitly use a transaction.

Provide **two SQL queries** with:

- Output
- Explanation of results

4. Neo4j: Graph Modelling and Queries (15%)

- Describe your graph model:
 - Node labels (e.g. Store, Product, Category)
 - Relationship types (must include **at least two types**)
- Include a visual example of a small subgraph

Provide **two Cypher queries** with:

- Output
- Explanation of what patterns they reveal

5. XML Report Generation (10%)

- Describe the structure of your XML:
 - Root and child elements
 - Attributes (if used)
- Your XML must include **aggregated information**, such as:
 - Number of products per store
 - Average or maximum price
- Include the full XML file (or in appendix)
- Explain how you ensured the XML is well-formed
- (Optional) Validate using a DTD

6. Infrastructure and Security Reflection (15%)

Discuss the following in the context of your pipeline:

- **Operating System**
Why Linux/UNIX is suitable for your setup
- **Data Durability**
Recommend a RAID level and explain why it fits your databases
- **Power Protection**
Role of a UPS in maintaining availability

- **Security**
 - How you stored credentials securely
 - Why SSH key authentication is more secure than passwords

7. Challenges and Reflection (20%)

- Describe specific issues you encountered, such as:
 - API limitations or errors
 - Handling nested JSON
 - Duplicate data
 - Transaction failures
- Explain how you resolved them
- Reflect on what you learned about integrating:
 - Document databases
 - Relational databases
 - Graph databases

Grading Summary

Report (50%)

Section	Weight
Environment and Tools	10%
MongoDB	15%
PostgreSQL	15%
Neo4j	15%
XML	10%
Infrastructure & Security	15%
Reflection	20%

Supporting Files (50%)

Component	Weight
Scripts functional and clear	25%
Queries correct and complete	20%
XML output valid	5%